

Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

УЧЕБНЫЕ МАТЕРИАЛЫ СЕМИНАРА №7
ОСНОВЫ ТЕСТИРОВАНИЯ В .NET

Учебные материалы по дисциплине
«Архитектура автоматизированных систем обработки
информации и управления»
Профиль: «Архитектура программных систем»

Москва – 2026

Оглавление

1	ЦЕЛЬ.....	4
2	ОСНОВЫ СВЯЗИ ПРИНЦИПОВ ТЕСТИРОВАНИЯ С ПРИНЦИПАМИ SOLID И ВНЕДРЕНИЕМ ЗАВИСИМОСТЕЙ.....	4
2.1	ПРИМЕР ХОРОШО И ПЛОХО ТЕСТИРУЕМОГО КОДА	5
2.2	ТЕСТЫ КАК ИСПОЛНЯЕМАЯ ДОКУМЕНТАЦИЯ	7
2.3	ЭКОНОМИЧЕСКАЯ МОТИВАЦИЯ ТЕСТИРОВАНИЯ.....	8
3	КЛАССИФИКАЦИЯ ТЕСТОВ И ПИРАМИДА ТЕСТИРОВАНИЯ	9
3.1	ЗАЧЕМ НУЖНА КЛАССИФИКАЦИЯ	9
3.2	ВИДЫ ТЕСТОВ ПО УРОВНЮ ИЗОЛЯЦИИ	9
3.3	ПИРАМИДА ТЕСТИРОВАНИЯ И АЛЬТЕРНАТИВНЫЕ МОДЕЛИ	10
3.4	ЧТО ТАКОЕ «ЮНИТ» (МОДУЛЬ) В МОДУЛЬНОМ ТЕСТИРОВАНИИ: КЛАССИЧЕСКАЯ И ЛОНДОНСКАЯ ШКОЛЫ	13
4	МОДУЛЬНОЕ ТЕСТИРОВАНИЕ НА C#: ФРЕЙМВОРКИ И БАЗОВЫЕ ПОДХОДЫ15	
4.1	СОЗДАНИЕ ТЕСТОВОГО ПРОЕКТА В ЭКОСИСТЕМЕ .NET	15
4.2	ТРИ ФРЕЙМВОРКА МОДУЛЬНОГО ТЕСТИРОВАНИЯ	17
4.3	СРАВНЕНИЕ ФРЕЙМВОРКОВ НА ПРИМЕРЕ	18
4.3.1	Фреймворк <i>MSTest</i>	19
4.3.2	Фреймворк <i>NUnit</i>	21
4.3.3	Фреймворк <i>xUnit</i>	22
4.4	ПАРАМЕТРИЗОВАННЫЕ ТЕСТЫ.....	22
4.5	ПОДГОТОВКА И ОЧИСТКА ДАННЫХ ДЛЯ ТЕСТА: SETUP И TEARDOWN.....	24
4.6	АНАТОМИЯ ТЕСТА: ПАТТЕРН AAA	27
4.7	СОГЛАШЕНИЯ ПО ИМЕНОВАНИЮ ТЕСТОВ	28
4.8	БИБЛИОТЕКИ АССЕРТОВ: SHOULDLY	28
4.9	ПРИНЦИПЫ ХОРОШЕГО ТЕСТА: FIRST	29
5	ТЕСТОВЫЕ ДУБЛЁРЫ И БИБЛИОТЕКА MOQ	30
5.1	ЗАЧЕМ НУЖНЫ ТЕСТОВЫЕ ДУБЛЁРЫ.....	30
5.2	КЛАССИФИКАЦИЯ МЕСЗАРОСА	30
5.3	РУЧНОЙ ТЕСТОВЫЙ ДУБЛЕР	30
5.4	СПЕЦИАЛИЗИРОВАННЫЕ МОС-БИБЛИОТЕКИ.....	32
6	РАЗРАБОТКА ЧЕРЕЗ ТЕСТИРОВАНИЕ (TDD).....	34
6.1	ЧТО ТАКОЕ TDD	34
6.2	ЦИКЛ RED — GREEN — REFACTOR	34
6.3	ТРИ ЗАКОНА TDD (РОБЕРТ МАРТИН)	35
6.4	ДЕМОНСТРАЦИЯ: РАЗРАБОТКА FAMILYFINERULE МЕТОДОМ TDD.....	35

6.4.1	<i>Итерация 1: льготный период</i>	36
6.4.2	<i>Итерация 2: оплата после льготного периода</i>	36
6.4.3	<i>Итоги</i>	37
6.5	Что даёт TDD КРОМЕ ТЕСТОВ.....	37
6.6	КОГДА TDD НЕУМЕСТЕН	37
7	ПОВЕДЕНЧЕСКИ-ОРИЕНТИРОВАННАЯ РАЗРАБОТКА (BDD)	38
7.1	Что такое BDD	38
7.2	Принцип работы BDD	38
8	ТЕСТИРОВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА	46
8.1	WinForms с FLAUI.....	46
8.2	WPF с FLAUI	47
8.3	ASP.NET Core MVC с Playwright	48
9	ОСНОВНЫЕ ВЫВОДЫ	49

1 Цель

Освоить практику автоматизированного тестирования программ на C#: научиться писать модульные тесты с использованием современных тестовых фреймворков, понять разработку через тестирование (TDD) и поведенчески-ориентированную разработку (BDD), познакомиться с другими видами тестирования, применяемыми в промышленной разработке. Понять связь принципов тестирования с принципами SOLID и внедрением зависимостей, изученными на предыдущих семинарах.

2 Основы связи принципов тестирования с принципами SOLID и внедрением зависимостей

На предыдущих занятиях мы научились разделять зависимости через интерфейсы и внедрять их через конструктор, а также распознавать нарушения принципов SOLID и применять рефакторинг. В обоих случаях оставался открытым один и тот же вопрос: как убедиться, что после внесённых изменений программа продолжает работать корректно?

Мартин Фаулер в книге «Рефакторинг» прямо формулирует ответ: «Без надёжного набора тестов рефакторинг превращается в опасное занятие». Любое изменение существующего кода — будь то смена реализации ILogger, выделение нового класса по принципу SRP или замена if/else полиморфизмом — потенциально может сломать поведение, которое программа должна сохранять. Единственный эффективный способ убедиться в работоспособности программы после рефакторинга — создать набор автоматизированных тестов, который можно запустить за секунды и получить однозначный ответ «работает / не работает».

Кроме того, тестируемость кода и качество его проектирования — это две стороны одного явления. Класс, написанный с нарушениями SRP и DIP, как правило, тяжело покрыть тестами; класс, спроектированный по

SOLID, тестируется тривиально. Это утверждение мы проверим в течение семинара.

2.1 Пример хорошо и плохо тестируемого кода

Рассмотрим две версии класса BonusCalculator, рассчитывающего премию сотрудника. Первая версия — «наивная»:

```
class BonusCalculatorTightlyCoupled
{
    public decimal Calculate(int employeeId)
    {
        // Зависимость 1: прямое обращение к базе данных
        using var conn = new
SqlConnection("Server=...;Database=HR;...");
        conn.Open();
        var cmd = new SqlCommand(
            "SELECT Salary, YearsOfExperience, IsFullTime FROM
Employees WHERE Id = @id", conn);
        cmd.Parameters.AddWithValue("@id", employeeId);
        // ... чтение результата ...

        // Зависимость 2: текущая дата берётся из системного
времени
        if (DateTime.Now.Month != 12)
            return 0m;

        // Зависимость 3: запись в файл журнала
        File.AppendAllText("bonus.log", $"Расчёт для
{employeeId}\n");

        // ... расчёт ...
        return 0m;
    }
}
```

Чтобы проверить логику расчёта премии в этом классе, требуется:

- развернуть базу данных с конкретной структурой таблиц;
- наполнить её тестовыми данными;
- ждать декабря (либо подменять системное время на уровне ОС);
- следить за тем, чтобы файл bonus.log существовал и был доступен на запись.

Тест становится медленным, нестабильным и зависит от внешнего окружения. На практике подобные классы обычно остаются непокрытыми тестами вообще.

Вторая версия спроектирована по принципам IoC/DI и SOLID:

```
class BonusCalculator
{
    private readonly IEmployeeRepository _repository;
    private readonly IClock _clock;
    private readonly ILogger _logger;

    public BonusCalculator(
        IEmployeeRepository repository, IClock clock, ILogger
        logger)
    {
        _repository = repository;
        _clock = clock;
        _logger = logger;
    }

    public decimal Calculate(int employeeId)
    {
        Employee employee = _repository.GetById(employeeId)
            ?? throw new ArgumentException($"Сотрудник
{employeeId} не найден");

        if (_clock.Now.Month != 12) return 0m;

        _logger.Log($"Расчёт премии для {employee.Name}");

        if (!employee.IsFullTime)
            return employee.Salary * 0.07m;

        return employee.YearsOfExperience > 5
            ? employee.Salary * 0.15m
            : employee.Salary * 0.12m;
    }
}
```

Тест для этого класса пишется в несколько строк кода и не требует ни базы данных, ни файловой системы, ни ожидания декабря — все три зависимости подменяются на тестовые объекты, эти подходы рассматриваются далее.

Главный вывод этого примера: **тестируемость — не отдельное свойство кода, а индикатор качества его проектирования.** Если класс

тяжело покрыть тестами, это почти всегда означает, что в нём нарушены принципы IoC/DI и SOLID. И наоборот: следование принципам автоматически даёт хорошо тестируемый код.

2.2 Тесты как исполняемая документация

У автоматизированных тестов есть вторая, не менее важная функция, помимо проверки корректности. Хорошо написанный тест — это пример использования класса, который не может устареть, потому что выполняется при каждой сборке.

Сравним два способа объяснения как работает FineCalculator. Первый вариант. Документация в виде комментария:

```
/// <summary>
/// Для пенсионеров первые 7 дней просрочки бесплатны,
/// далее по 3 рубля в день.
/// </summary>
class SeniorFineRule : FineRule { /* ... */ }
```

Этот комментарий может быть уже неактуален, это невозможно проверить.

Второй вариант. Документация в форме теста:

```
[Fact]
public void Senior_OverdueByFirstSevenDays_NoFine()
{
    var rule = new SeniorFineRule();
    Assert.Equal(0m, rule.Calculate(daysOverdue: 7));
}
```

Этот комментарий в форме теста гарантированно соответствует текущему поведению кода — иначе выполнение теста не пройдёт. Если правила изменятся, тесты упадут, и разработчик их обновит. Такой подход к документированию поведения системы лежит в основе TDD и BDD фреймворков.

Кент Бек в книге «Экстремальное программирование. Разработка через тестирование» формулирует этот принцип так:

«Автоматизированные тесты — единственная форма документации, которой можно доверять».

2.3 Экономическая мотивация тестирования

В литературе по программной инженерии распространено наблюдение о росте стоимости устранения дефекта по мере его продвижения по этапам разработки. Эмпирические данные впервые были собраны Барри Бозмом в работе «Инженерное проектирование программного обеспечения» и затем уточнялись в других работах.

Конкретные числа в разных источниках различаются, но качественная картина устойчиво воспроизводится: ошибка, найденная при написании кода, обходится дешевле, чем найденная при ручном тестировании; найденная при ручном тестировании — дешевле, чем найденная пользователем в продакшене.

Модульный тест ловит ошибку в самом начале «жизненного цикла» приложения — за секунды после её появления, прямо в IDE разработчика. Чем большая часть дефектов обнаруживается именно так, тем дешевле обходится разработка системы в целом.

К этому добавляется второй эффект — психологический: разработчик, окружённый тестами, не боится менять код. Если рефакторинг сломает что-то важное, тесты немедленно об этом сообщат, и изменение можно откатить. Без тестов каждый рефакторинг — это риск, и накопленный страх перед изменениями приводит к тому, что код обрастает «защитными» костылями вместо того, чтобы упрощаться.

3 Классификация тестов и пирамида тестирования

3.1 Зачем нужна классификация

Когда говорят, что нужно «покрыть систему тестами», за этой фразой скрываются принципиально разные виды тестов с разной стоимостью, скоростью и областью применения. Один тест проверяет, что метод `CalculateEmployeeBonus` возвращает 0 для уволенного сотрудника; другой — что вся система от веб-формы до базы данных корректно обрабатывает заявку. Это разные задачи, для них существуют разные инструменты, и смешение видов тестов приводит к путанице и неэффективным тестовым наборам.

В этом разделе мы рассмотрим базовую классификацию и каноническую модель распределения тестов в проекте — пирамиду тестирования.

3.2 Виды тестов по уровню изоляции

Тесты принято классифицировать по тому, какую часть системы они проверяют и насколько изолированы от внешних зависимостей.

Модульный тест (unit test). Проверяет один «юнит» кода — обычно один класс или один метод — в изоляции от его зависимостей. Все обращения к базе данных, файловой системе, сети, текущему времени и прочим внешним ресурсам подменяются тестовыми дублёрами. Тест выполняется за миллисекунды, не имеет внешних предусловий, может быть запущен в любой момент на любой машине.

Пример: проверка, что `StudentFineRule.Calculate(3)` возвращает `6m` — это модульный тест. В нём нет ни сети, ни файлов, ни базы данных.

Интеграционный тест (integration test). Проверяет совместную работу нескольких компонентов или взаимодействие приложения с реальной внешней системой — настоящей (или приближённой к

настоящей) базой данных, файловой системой, очередью сообщений. Выполняется заметно медленнее модульного — миллисекунды превращаются в десятки и сотни миллисекунд.

Пример: проверка, что `EmployeeRepository.Save(employee)` действительно записывает строку в SQLite-базу, а `GetById` её затем считывает.

Сквозной тест (end-to-end, E2E). Проверяет работу системы целиком — от внешнего интерфейса до внешнего интерфейса. Для веб-приложения это означает запуск браузера, переходы по страницам, заполнение форм и проверку результата. E2E-тесты медленные (секунды и десятки секунд), хрупкие (ломаются от мелких изменений UI) и требуют сложной инфраструктуры.

Пример: открыть в браузере страницу библиотеки, авторизоваться, выдать книгу читателю, проверить, что в его карточке появилась запись о выдаче.

В разных источниках используются разные названия и существуют дополнительные категории — компонентные тесты, контрактные тесты, smoke-тесты, регрессионные тесты, — но базовое деление на три уровня является общепринятым.

3.3 Пирамида тестирования и альтернативные модели

Распределение тестов по уровням предложил Майк Кон в книге «Scrum. Гибкая разработка ПО». Эту идею подробно разобрал и популяризировал Мартин Фаулер в статье «TestPyramid» 2012 года, <https://martinfowler.com/bliki/TestPyramid.html>

Пирамида состоит из трёх уровней:



Рис. 1. Пирамида тестирования.

Принципы, которые формулирует пирамида:

- Чем выше уровень — тем меньше должно быть тестов. Не «много тестов — хорошо», а «правильно распределенные тесты — хорошо».
- Чем ниже уровень — тем быстрее и стабильнее тесты. Тысяча модульных тестов выполняется за пару секунд; тысяча E2E-тестов — за час, и часть из них может упасть по случайным причинам.
- Каждый уровень покрывает то, что не могут покрыть более низкие уровни. Бессмысленно проверять формулу расчёта штрафа через E2E-тест с запуском браузера — это работа модульного теста. И наоборот, бессмысленно проверять корректную авторизацию пользователя серией модульных тестов на внутренние классы — нужен интеграционный или E2E.

Типичная «антипаттерн» тестирования проекта — «перевернутая пирамида» (или, как её называет Фаулер, «test ice cream cone»): много медленных E2E-тестов, мало интеграционных, почти нет модульных. Сборка длится часами, тесты постоянно «мигают» (то падают, то проходят

без видимых изменений в коде), команда теряет к ним доверие и в итоге отключает.

Пирамида Кона — не единственная модель. В современной литературе обсуждаются альтернативы, отражающие специфику отдельных классов систем.

Одним из известных примеров является «Testing Trophy» — модель «кубок» или «трофей», предложенная Кентом К. Доддсом для фронтенд- и full-stack-разработки (статья «The Testing Trophy and Testing Classifications», 2021, <https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications>).

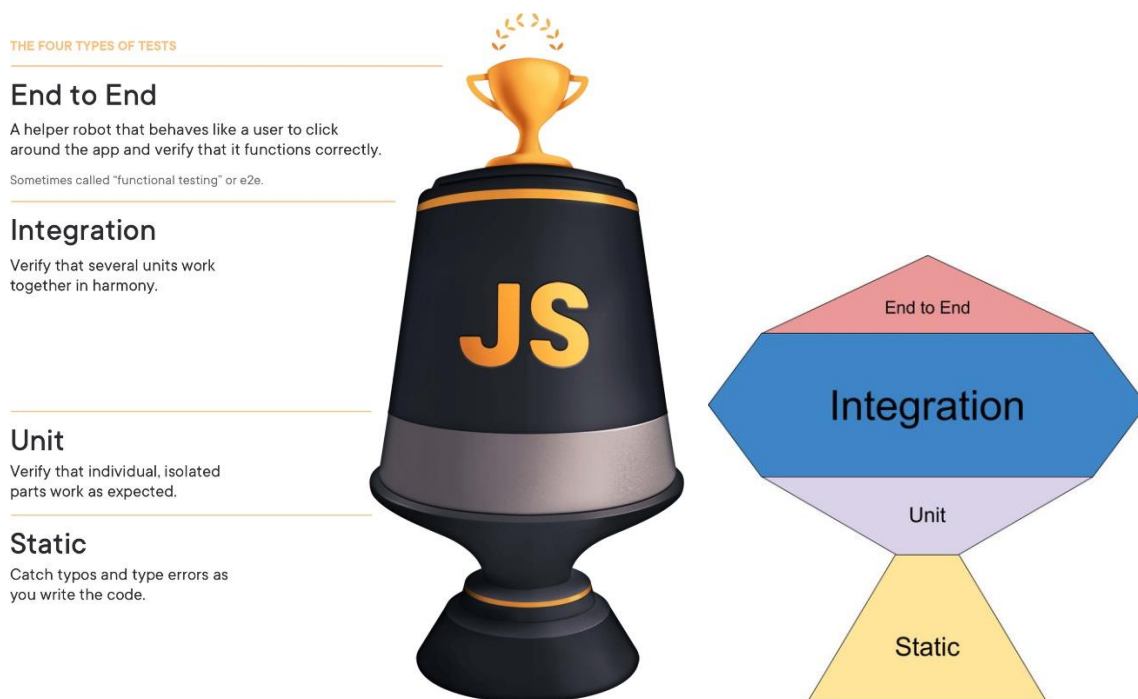


Рис. 2. Модель «кубок» (изображение из статьи Кента К. Доддса).

«Трофей» переносит акцент с модульных тестов на интеграционные: для веб-приложений, где много логики связано со взаимодействием компонентов, основная масса тестов располагается на интеграционном уровне, а не на модульном.

Важно, что альтернативные модели не отменяют пирамиду, а уточняют её для конкретных классов задач. Для типичного бизнес-

приложения с богатой бизнес-логикой предметной области (например, с расчётом штрафов и премий, на котором мы сосредоточены в этом семинаре) пирамида Кона остаётся актуальной моделью по умолчанию.

3.4 Что такое «юнит» (модуль) в модульном тестировании: классическая и лондонская школы

При обсуждении модульного тестирования возникает закономерный вопрос: что считать тем самым «юнитом», который тестируется в изоляции? Один метод? Один класс? Группу совместно работающих классов?

Исторически сложились два ответа на этот вопрос, известные как классическая школа и лондонская (моккистская) школа.

Классическая школа ведёт начало от Кента Бека. Юнит — это единица поведения, обычно один класс или небольшая группа классов, тесно связанных по смыслу. Зависимости от внешнего мира (базы данных, файловой системы, сети) подменяются на тестовые дублёры. Внутренние «соавторы», не имеющие побочных эффектов, остаются настоящими — тестируется именно совместная работа. Эту школу также называют «детройтской», поскольку Бек в своё время работал в Детройте над проектом в компании Chrysler.

Лондонская школа связана с книгой Стива Фримена и Ната Прайса «Growing Object-Oriented Software, Guided by Tests» (Addison-Wesley, 2009). Юнит — это всегда один класс, и при тестировании подменяются на моки (заглушки) все его зависимости — даже если это собственные классы того же приложения. Каждый тест проверяет один класс в полной изоляции от любых других классов системы.

Различия проявляются на уровне дизайна тестов. Сравним подход к тесту BonusCalculator для случая «у сотрудника зарплата 100 000, опыт > 5 лет, полная занятость, декабрь — премия 15 %».

Классический стиль:

```
[Fact]
public void Senior_FullTime_December_Bonus15Percent()
{
    var employee = new Employee
    {
        Id = 1,
        Name = "Иванов",
        Salary = 100_000m,
        YearsOfExperience = 7,
        IsFullTime = true
    };
    var repository = new InMemoryEmployeeRepository(new[] {
employee });
    var clock = new FixedClock(new DateTime(2025, 12, 15));
    var calculator = new BonusCalculator(repository, clock, new
NullLogger());

    decimal bonus = calculator.Calculate(employeeId: 1);

    Assert.Equal(15_000m, bonus);
}
```

В этом тесте InMemoryEmployeeRepository — это настоящий рабочий класс, а не мок. Тестируется именно поведение системы.

Лондонский стиль:

```
[Fact]
public void Senior_FullTime_December_Bonus15Percent()
{
    var employee = new Employee { /* ... */ };
    var repositoryMock = new Mock<IEmployeeRepository>();
    repositoryMock.Setup(r => r.GetById(1)).Returns(employee);
    var clockMock = new Mock<IClock>();
    clockMock.Setup(c => c.Now).Returns(new DateTime(2025, 12,
15));
    var loggerMock = new Mock<ILogger>();

    var calculator = new BonusCalculator(
        repositoryMock.Object, clockMock.Object,
        loggerMock.Object);

    decimal bonus = calculator.Calculate(employeeId: 1);

    Assert.Equal(15_000m, bonus);
    repositoryMock.Verify(r => r.GetById(1), Times.Once);
}
```

Все связанные классы имитируются в виде Mock'ов (заглушек).

Подробное сравнение двух школ дано Мартином Фаулером в статье «Mocks Aren't Stubs» (<https://martinfowler.com/articles/mocksArentStubs.html>). Сообщество пришло к мнению, что классическая школа в среднем приводит к более устойчивым к рефакторингу тестам. Мы также будем пользоваться преимущественно классическим подходом — подменять только «настоящие» внешние зависимости (БД, время, файлы, сеть), а внутренние классы оставлять реальными там, где это не приводит к разрастанию теста.

4 Модульное тестирование на C#: фреймворки и базовые подходы

4.1 Создание тестового проекта в экосистеме .NET

В .NET модульные тесты пишутся в отдельном проекте — это делается для того, чтобы тестовая сборка не попадала в реальную исполняемую среду, и чтобы тестовые зависимости (Moq, FluentAssertions и т. п.) не загрязняли основное приложение. Виды тестовых проектов приведены на следующей форме создания проекта:

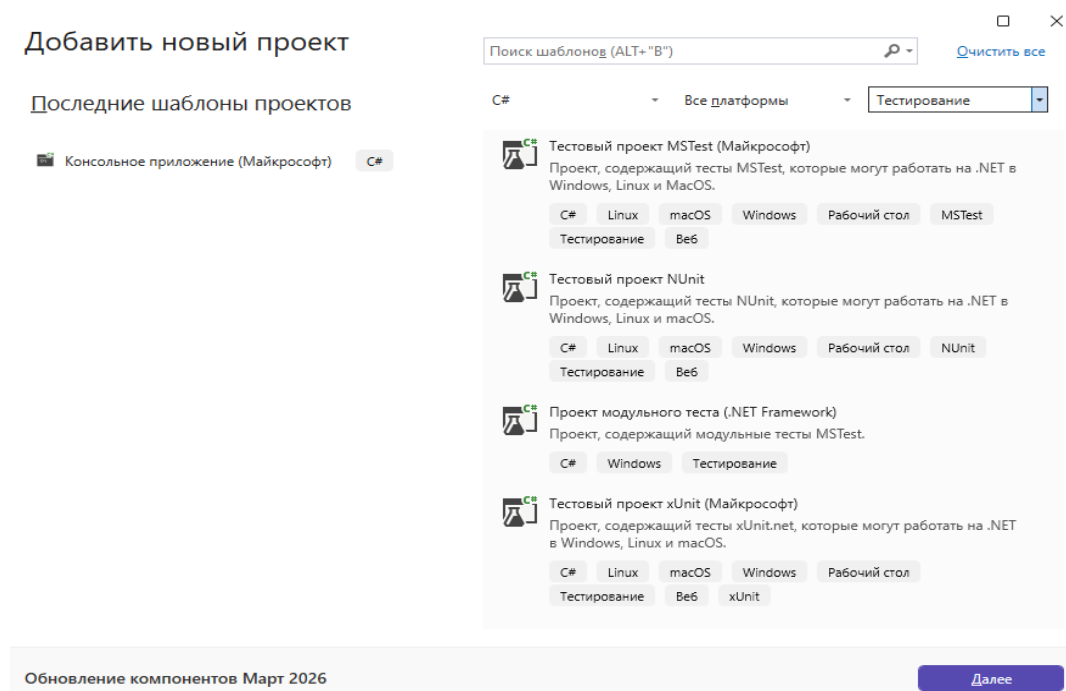


Рис. 3. Виды тестовых проектов (для фреймворков MSTest, NUnit, xUnit.net).

При создании проектов будем использовать значения по умолчанию.

Дополнительные сведения

Тестовый проект MSTest (Майкрософт)

C# Linux macOS Windows Рабочий стол MSTest Тестирование Веб

Платформа ⓘ

.NET 10.0 (долгосрочная поддержка)

☐ Использовать MSTest.Sdk ⓘ

Средство выполнения тестов ⓘ

VSTest

Средство оценки объема протестированного кода ⓘ

Microsoft.CodeCoverage

Средство ⓘ

Отсутствует

Обновление компонентов Март 2026

Назад

Создать

Рис. 4. Параметры создания проекта MSTest.

Дополнительные сведения

Тестовый проект NUnit

C# Linux macOS Windows Рабочий стол NUnit Тестирование Веб

Платформа ⓘ

.NET 10.0 (долгосрочная поддержка)

Обновление компонентов Март 2026

Назад

Создать

Рис. 5. Параметры создания проекта NUnit.

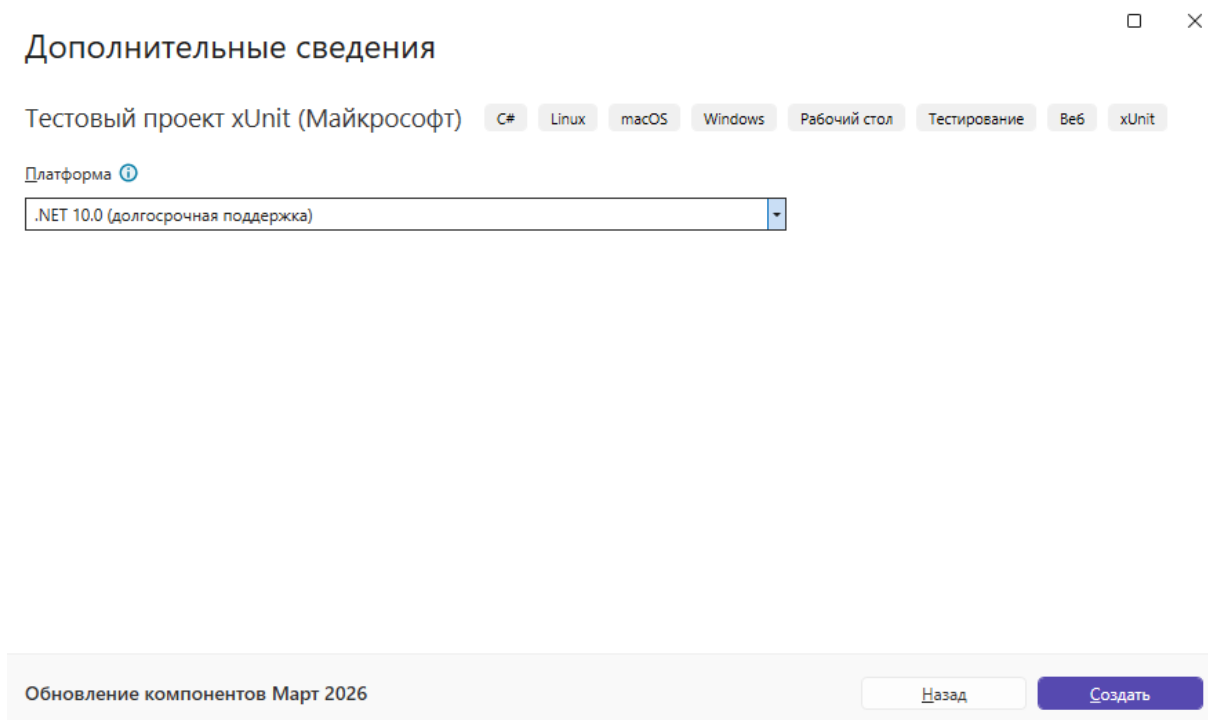


Рис. 6. Параметры создания проекта xUnit.

4.2 Три фреймворка модульного тестирования

В экосистеме .NET сложились три основных фреймворка модульного тестирования, и все три активно поддерживаются. Приведём их краткую характеристику.

MSTest — фреймворк от Microsoft, исторически встроенный в Visual Studio. Современная версия опубликована как open-source-проект на GitHub под лицензией MIT (<https://github.com/microsoft/testfx>). Для команд, работающих в инфраструктуре Microsoft, это часто фреймворк «по умолчанию».

NUnit — портированный с языка Java JUnit, появился в 2002 году. Один из его создателей — Джим Ньюкирк, ранее участник команды JUnit. Текущая версия использует репозиторий: <https://github.com/nunit/nunit>.

xUnit.net — создан в 2007 году Брэдом Уилсоном и тем же Джимом Ньюкирком как «переосмысление» NUnit с учётом накопленного опыта.

Главные идеи: использовать стандартные средства C# (конструктор, IDisposable) вместо специальных атрибутов, обеспечивать строгую изоляцию каждого теста, минимизировать количество понятий в API. Репозиторий: <https://github.com/xunit/xunit>. Это фреймворк по умолчанию в шаблонах ASP.NET Core и широко используется в самих проектах Microsoft. В настоящее время этот фреймворк чаще всего выбирается разработчиками сообщества .NET

4.3 Сравнение фреймворков на примере

Сравним три фреймворка на одной и той же простой задаче. Добавим в исходный тестируемый консольный проект следующий класс:

```
public class RegularFineRule
{
    public decimal Calculate(int daysOverdue) => daysOverdue * 5m;
}
```

Для того чтобы осуществлять тестирование необходимо в каждом из трех тестовых проектов (для каждого фреймворка) создать ссылку на тестируемый консольный проект.

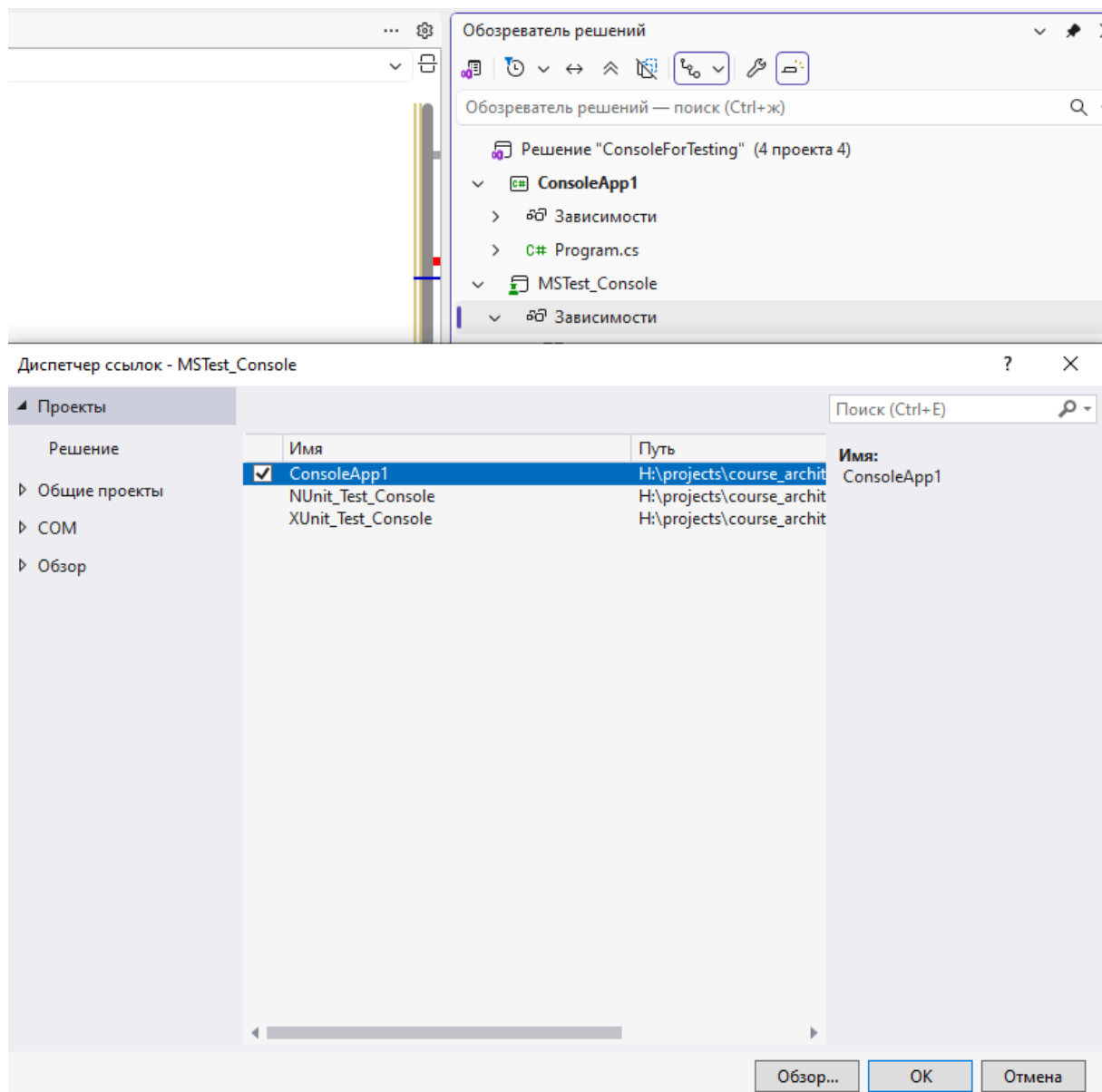


Рис. 7. Создание ссылки на тестируемый консольный проект.

4.3.1 Фреймворк MSTest

В файле «Test1.cs» проекта «MSTest_Console» разместим следующий код:

```
// Чтобы были видны классы тестируемого проекта
using ConsoleApp1;

namespace MSTest_Console
{
    [TestClass]
    public sealed class Test1
    {
        [TestMethod]
        public void Calculate_ThreeDaysOverdue_Returns15()
        {
            // Arrange
            var rule = new RegularFineRule();

            // Act
            decimal fine = rule.Calculate(daysOverdue: 3);

            // Assert
            Assert.AreEqual(15m, fine);
        }
    }
}
```

Для запуска тестов необходимо нажать правую кнопку на проекте и выбрать пункт меню «Выполнить тесты».

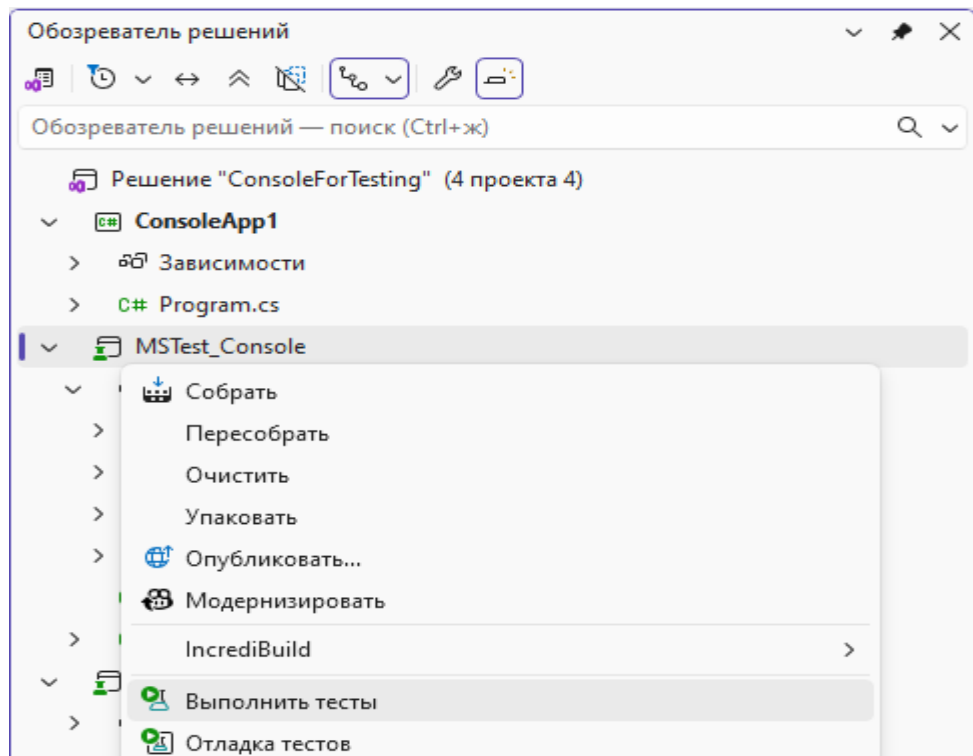


Рис. 8. Запуск тестов.

После этого откроется отдельное окно обозревателя тестов, в котором можно увидеть результаты успешного выполнения тестов.

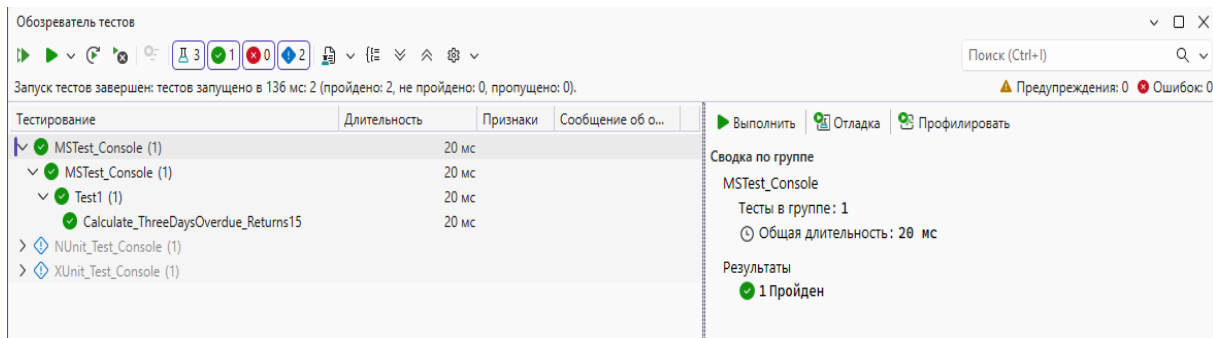


Рис. 9. Обозреватель тестов (успешное выполнение теста).

Заменим в строке кода «Assert.AreEqual(15m, fine);» значение 15m на значение 16m и выполним тесты повторно. В этом случае происходит «падение» теста, так как вычисленное значение не совпадает с эталонным тестовым значением.

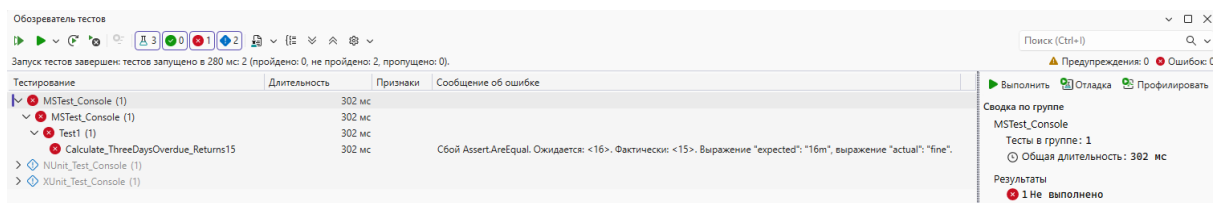


Рис. 10. Обозреватель тестов («падение» теста).

4.3.2 Фреймворк NUnit

В файле «UnitTest1.cs» проекта «NUnit_Test_Console» разместим следующий код:

```
// Чтобы были видны классы тестируемого проекта
using ConsoleApp1;

namespace NUnit_Test_Console
{
    public class Tests
    {
        [Test]
        public void Calculate_ThreeDaysOverdue_Returns15()
        {
        }
    }
}
```

```

        // Arrange
        var rule = new RegularFineRule();

        // Act
        decimal fine = rule.Calculate(daysOverdue: 3);

        // Assert
        Assert.That(fine, Is.EqualTo(15m));
    }
}

```

4.3.3 Фреймворк xUnit

В файле «UnitTest1.cs» проекта «XUnit_Test_Console» разместим следующий код:

```

using ConsoleApp1;

namespace XUnit_Test_Console
{
    public class UnitTest1
    {
        [Fact]
        public void Calculate_ThreeDaysOverdue_Returns15()
        {
            // Arrange
            var rule = new RegularFineRule();

            // Act
            decimal fine = rule.Calculate(daysOverdue: 3);

            // Assert
            Assert.Equal(15m, fine);
        }
    }
}

```

4.4 Параметризованные тесты

Один из самых частых сценариев в юнит-тестировании — проверка одного и того же метода на нескольких наборах данных. Все три фреймворка поддерживают эту возможность, но через разные конструкции.

Пример для MSTest:

```

[TestMethod]
[DataRow(0, "0")]
[DataRow(1, "5")]
[DataRow(3, "15")]
[DataRow(10, "50")]
public void Calculate_VariousDays_ReturnsCorrectFine(int days,
string expectedStr)
{
    // Преобразуем строку в decimal
    decimal expected = decimal.Parse(expectedStr,
System.Globalization.CultureInfo.InvariantCulture);
    var rule = new RegularFineRule();
    Assert.AreEqual(expected, rule.Calculate(days));
}

```

Пример для NUnit:

```

[TestCase(0, 0)]
[TestCase(1, 5)]
[TestCase(3, 15)]
[TestCase(10, 50)]
public void Calculate_VariousDays_ReturnsCorrectFine(int days,
decimal expected)
{
    var rule = new RegularFineRule();
    Assert.That(rule.Calculate(days), Is.EqualTo(expected));
}

```

Пример для xUnit:

```

[Theory]
[InlineData(0, 0)]
[InlineData(1, 5)]
[InlineData(3, 15)]
[InlineData(10, 50)]
public void Calculate_VariousDays_ReturnsCorrectFine(int days,
decimal expected)
{
    var rule = new RegularFineRule();
    Assert.Equal(expected, rule.Calculate(days));
}

```

В xUnit для отделения параметризованных тестов от обычных введён отдельный атрибут [Theory] (вместо [Fact]). Это сознательное решение: «факт» утверждает что-то конкретное, «теория» утверждает что-то для класса входных данных. NUnit и MSTest используют единый атрибут для обоих случаев.

В обозревателе тестов параметризованный тест отображается как одна строка с возможностью «развернуть» в отдельные подтесты — это удобно для диагностики: видно, какой именно набор данных привёл к падению.

4.5 Подготовка и очистка данных для теста: *setup* и *teardown*

В реальных тестах обычно требуется некоторая подготовка перед каждым тестом (создать объекты, открыть соединение) и очистка после (закрыть соединение, удалить временный файл). Здесь различия фреймворков становятся принципиальными.

Пример для MSTest:

```
[TestClass]
public class FineCalculatorTests
{
    private FineCalculator _calculator;

    [TestInitialize]                // Перед КАЖДЫМ тестом
    public void Setup()
    {
        _calculator = new FineCalculator(new RegularFineRule());
    }

    [TestCleanup]                  // После КАЖДОГО теста
    public void Cleanup() { /* ... */ }

    [ClassInitialize]              // ОДИН РАЗ перед всеми тестами
    класса
    public static void ClassSetup(TestContext context) { /* ... */ }

    [ClassCleanup]                 // ОДИН РАЗ после всех тестов класса
    public static void ClassCleanup() { /* ... */ }

    [TestMethod]
    public void Calculate_ThreeDays_Returns15()
    {
        Assert.AreEqual(15m, _calculator.Calculate(3));
    }
}
```


Пример для NUnit:

```

[TestFixture]
public class FineCalculatorTests
{
    private FineCalculator _calculator;

    [SetUp] // Перед каждым тестом
    public void Setup()
    {
        _calculator = new FineCalculator(new RegularFineRule());
    }

    [TearDown] // После каждого теста
    public void TearDown() { /* ... */ }

    [OneTimeSetUp] // Один раз перед всеми
тестами
    public void OneTimeSetup() { /* ... */ }

    [OneTimeTearDown] // Один раз после всех тестов
    public void OneTimeTearDown() { /* ... */ }

    [Test]
    public void Calculate_ThreeDays_Returns15()
    {
        Assert.That(_calculator.Calculate(3), Is.EqualTo(15m));
    }
}

```

Пример для xUnit

```

public class FineCalculatorTests : IDisposable
{
    private readonly FineCalculator _calculator;

    public FineCalculatorTests() // Перед каждым тестом
    {
        _calculator = new FineCalculator(new RegularFineRule());
    }

    public void Dispose() { /* После каждого теста */ }

    [Fact]
    public void Calculate_ThreeDays_Returns15()
    {
        Assert.Equal(15m, _calculator.Calculate(3));
    }
}

```

```
// Для setup один раз на класс – отдельный механизм через
// IClassFixture<T>
public class DatabaseFixture : IDisposable
{
    public DatabaseFixture() { /* инициализация один раз */ }
    public void Dispose() { /* очистка один раз */ }
}

public class FineCalculatorIntegrationTests
    : IClassFixture<DatabaseFixture>, IDisposable
{
    private readonly DatabaseFixture _fixture;
    public FineCalculatorIntegrationTests(DatabaseFixture
fixture)
        => _fixture = fixture;

    public void Dispose() { /* per-test cleanup */ }

    // ... тесты ...
}
```

В xUnit подход к подготовке данных (Setup) и очистке (Teardown) сильно отличается от MSTest или NUnit. Здесь нет атрибутов [TestInitialize] или [SetUp]. Вместо них используется стандартный жизненный цикл класса в C#.

В первом блоке кода xUnit создает новый экземпляр класса FineCalculatorTests для каждого отдельного теста ([Fact] или [Theory]).

- Конструктор (public FineCalculatorTests()): Это замена [TestInitialize]. Поскольку для каждого теста создается новый объект класса, код в конструкторе гарантированно выполнится перед каждым тестом. Это обеспечивает изоляцию: тесты не влияют друг на друга через общее состояние _calculator.
- Интерфейс IDisposable: Если вам нужно что-то очистить после теста (замена [TestCleanup]), вы реализуете метод Dispose(). xUnit вызовет его сразу после завершения теста.

Если создание объекта (например, подключение к базе данных) занимает долгое время, вы не хотите делать это перед каждым тестом. Для этого используется Fixtures.

`DatabaseFixture`: Это вспомогательный класс. `xUnit` создаст его один раз перед запуском самого первого теста в классе и удалит только тогда, когда все тесты в этом классе отработают.

`IClassFixture<DatabaseFixture>`: Этот интерфейс — «метка» для `xUnit`. Он говорит: «Пожалуйста, создай один экземпляр `DatabaseFixture` и пробрось его в конструктор моего тестового класса».

В `JUnit` поведение по умолчанию таково, что один и тот же объект класса используется всеми тестовыми методами. Это означает, что поля, изменённые в одном тесте, могут «протечь» в другой. Поэтому в `JUnit` `setup`-метод обычно сбрасывает состояние перед каждым тестом. В `xUnit` изоляция гарантируется самой моделью: новый экземпляр класса это чистое состояние. Этот выбор Брэд Уилсон обсуждает как «один из ключевых уроков, извлечённых из `JUnit`».

4.6 *Анатомия теста: паттерн ААА*

Каждый тест, независимо от фреймворка, имеет одинаковую структуру из трёх блоков. Этот паттерн известен как ААА — `Arrange` / `Act` / `Assert` (подготовка / действие / проверка). Описан Биллом Уэйком в статье «3А — `Arrange`, `Act`, `Assert`» и стал общепринятым стандартом.

Правила, которые делают ААА-тест читаемым:

- Один `Act` на тест. Если в тесте два действия — это два теста.
- Видимая граница между блоками. Допустимо разделять их пустыми строками или комментариями `// Arrange`, `// Act`, `// Assert`.
- Никакой логики в `Assert`. В блоке `Assert` не должно быть `if`, `for`, вычислений — только сравнение фактического результата с ожидаемым.

Альтернативное название того же паттерна, пришедшее из `BDD`, — `Given` / `When` / `Then` (Дано / Когда / Тогда). Это синонимы; разница только в терминологии.

4.7 Соглашения по именованию тестов

Тестов в проекте обычно сотни, и их чтение является постоянной задачей. Имя теста должно отвечать на три вопроса: что тестируется, в какой ситуации, с каким ожидаемым результатом. Необходимо отметить, что длинные имена тестовых методов считаются нормой, они пишутся один раз и потом как правило только читаются, поэтому понятность имен тестов важна.

Наиболее распространённое соглашение, рекомендуемое экспертами Microsoft (<https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices>):

«Действие_Сценарий_ОжидаемыйРезультат»

Примеры для класса FineCalculator:

```
public void Calculate_StudentOverdueByThreeDays_Returns6Rubles() { }
public void Calculate_StaffOverdueByAnyDays_ReturnsZero() { }
public void Calculate_SeniorOverdueByTenDays_ReturnsNineRubles() { }
public void Calculate_NegativeDaysOverdue_ThrowsArgumentException() {}
```

4.8 Библиотеки ассертов: Shouldly

Встроенные ассерты всех трёх фреймворков работают успешно, но при усложнении проверок их синтаксис становится тяжеловесным:

```
// Встроенные ассерты xUnit
Assert.NotNull(result);
Assert.Equal(3, result.Count);
Assert.All(result, item => Assert.True(item.IsValid));
Assert.Contains(result, item => item.Name == "Иванов");
```

Существуют библиотеки fluent-ассертов, которые делают то же самое в виде цепочки методов, читающейся как фраза на английском. Одной из таких библиотек является Shouldly (<https://github.com/shouldly/shouldly>).

Пример:

```
result.ShouldNotBeNull();
result.Count.ShouldBe(3);
result.ShouldAllBe(item => item.IsValid);
result.ShouldContain(item => item.Name == "Иванов");
```

4.9 Принципы хорошего теста: *FIRST*

Аббревиатуру FIRST для пяти ключевых свойств модульных тестов сформулировали Тим Оттингер и Джефф Лангр (статья «FIRST Properties of Unit Tests», 2007 г.).

- F — Fast (быстрый). Один модульный тест выполняется за миллисекунды. Тысяча тестов — за пару секунд. Если тесты идут минуты, разработчики перестают их запускать.
- I — Isolated / Independent (изолированный). Тест не зависит ни от других тестов, ни от внешнего состояния. Любой тест должен корректно отработать в любом порядке и в любом подмножестве. Падение теста A не должно ломать тест B.
- R — Repeatable (повторяемый). Запуск теста дважды на одних и тех же данных даёт одинаковый результат. Никаких зависимостей от текущего времени, генератора случайных чисел без фиксированного seed, доступа к интернету.
- S — Self-validating (самопроверяющийся). Результат теста — однозначное «прошёл / не прошёл». Никаких сообщений в консоль, которые разработчик должен проверять дополнительно. Если в тесте есть `Console.WriteLine` для проверки результата — это не тест, это эксперимент.
- T — Timely (своевременный). Тесты пишутся вовремя — в идеале до или одновременно с тестируемым кодом, а не «когда-нибудь потом». Накапливающийся долг непокрытого кода погасить впоследствии практически невозможно.

Иногда последнюю букву расшифровывают как Thorough (тщательный) — «тесты должны покрывать не только нормальные сценарии, но и граничные случаи, ошибки, нетипичные входы». Это ортогональное FIRST-свойство, и обе трактовки часто приводят вместе.

5 Тестовые дублёры и библиотека Moq

5.1 Зачем нужны тестовые дублёры

Большинство классов в реальных проектах имеют зависимости — на базу данных, файловую систему, текущее время, внешние сервисы. Если в тесте передавать настоящие реализации, тест перестаёт быть модульным: он становится медленным, нестабильным и зависит от внешнего состояния. Принципы FIRST нарушаются.

Решение — заменить настоящие зависимости на специальные объекты, имитирующие нужное поведение. Эти объекты называют тестовыми дублёрами (test doubles) — термин Джерарда Месзароса из книги «xUnit Test Patterns».

5.2 Классификация Месзароса

Выделяют пять видов дублёров. Подробный разбор — в статье Мартина Фаулера «Mocks Aren't Stubs» (<https://martinfowler.com/articles/mocksArentStubs.html>):

- Dummy – Передаётся, но не используется (например, NullLogger)
- Stub – Возвращает заранее заданные ответы
- Fake – Работающая упрощённая реализация
- Mock – Проверяет факт и параметры вызовов
- Spy – Записывает вызовы для последующей проверки

Часто все пять видов часто называют «моками», и библиотеки называются mocking-библиотеками — но при чтении чужого кода важно различать: Stub возвращает данные, Mock проверяет вызовы.

5.3 Ручной тестовый дублер

Простейший вариант такого дублера — это обычный класс, реализующий определенный интерфейс.

```
// Stub – возвращает фиксированное значение
public class StubClock : IClock
{
    private readonly DateTime _fixedTime;
    public StubClock(DateTime fixedTime) => _fixedTime = fixedTime;
    public DateTime Now => _fixedTime;
}

// Stub – всегда возвращает одного и того же сотрудника
public class StubEmployeeRepository : IEmployeeRepository
{
    private readonly Employee? _employee;
    public StubEmployeeRepository(Employee? e) => _employee = e;
    public Employee? GetById(int id) => _employee;
}

// Dummy – ничего не делает (он же – Null Object)
public class NullLogger : ILogger
{
    public void Log(string message) { }
}
```

Пример модульного теста с ручными дублерами:

```
[Fact]
public void Calculate_NotDecember_ReturnsZero()
{
    // Arrange
    var employee = new Employee
    {
        Id = 1,
        Name = "Иванов",
        Salary = 100_000m,
        YearsOfExperience = 7,
        IsFullTime = true
    };
    var calculator = new BonusCalculator(
        new StubEmployeeRepository(employee),
        new StubClock(new DateTime(2025, 6, 15)), // не декабрь
        new NullLogger());

    // Act
    decimal bonus = calculator.Calculate(employeeId: 1);

    // Assert
    Assert.Equal(0m, bonus);
}
```

Ручные дублеры предельно прозрачны и не требуют сторонних библиотек, но для сложных сценариев их написание становится утомительным, и тогда начинают использовать специализированные библиотеки.

5.4 Специализированные Мос-библиотеки

В .NET наиболее распространённая библиотека для автоматического создания дублёров — Moq (<https://github.com/devlooped/moq>). Альтернативы — NSubstitute (<https://nsubstitute.github.io/>) и FakeItEasy (<https://fakeiteasy.github.io/>) — функционально эквивалентны, отличаются синтаксисом.

Замечание о доверии к Moq. В августе 2023 года в Moq 4.20.0 был встроен компонент SponsorLink, без уведомления отправлявший хеш email-адреса разработчика на сторонний сервер. Уже через неделю в версии 4.20.2 SponsorLink был удалён, но часть команд после этого инцидента перешла на NSubstitute. Технически Moq остаётся работающей библиотекой; обзор событий — в архиве issue: <https://github.com/devlooped/moq/issues/1374>.

Библиотека Мос может быть установлена как nuget-пакет через Обзорщик решений или через командную строку. Пример команды: «dotnet add package Moq».

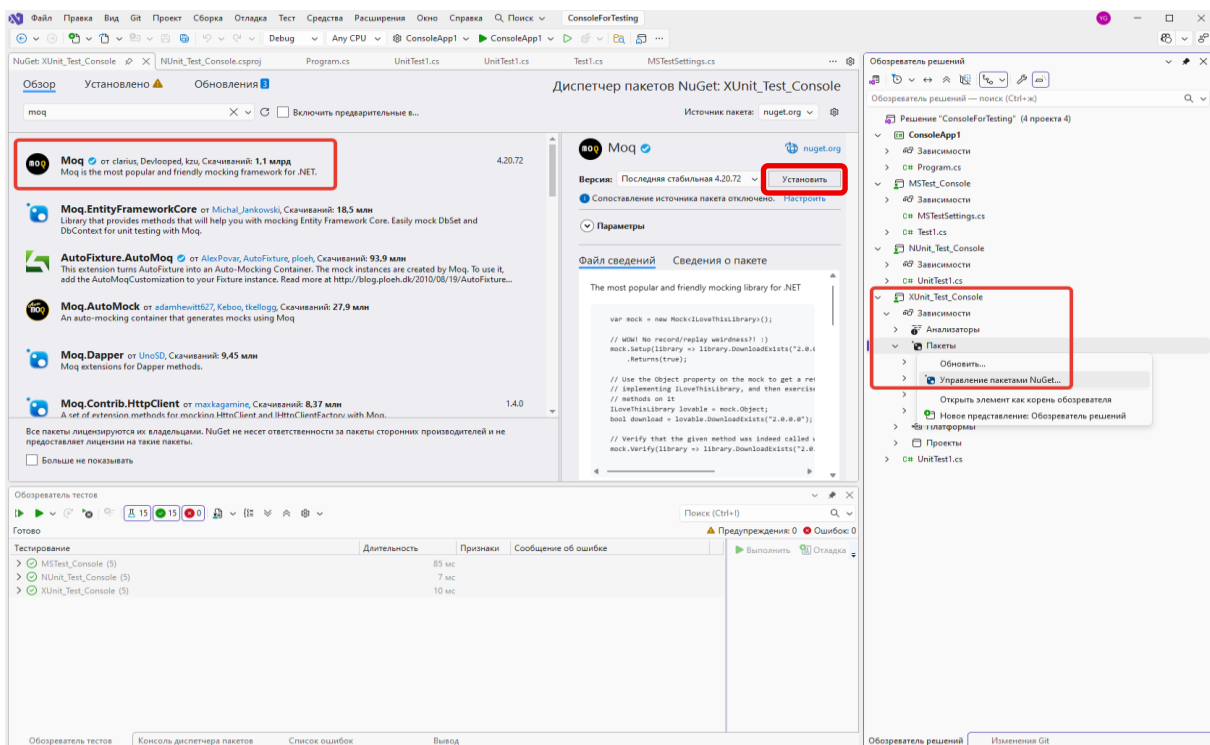


Рис. 11. Установка библиотеки Мос через Обзорщик решений.

Пример тестирования с использованием Moq:

```
using Moq;
using Xunit;

[Fact]
public void
Calculate_FullTimeSeniorInDecember_Returns15PercentAndLogsEvent(
)
{
    // Arrange
    var employee = new Employee
    {
        Id = 1,
        Name = "Иванов",
        Salary = 100_000m,
        YearsOfExperience = 7,
        IsFullTime = true
    };

    // Создаём моки
    var repositoryMock = new Mock<IEmployeeRepository>();
    var clockMock = new Mock<IClock>();
    var loggerMock = new Mock<ILogger>();

    // Настраиваем возвращаемые значения (роль Stub)
    repositoryMock.Setup(r => r.GetById(1)).Returns(employee);
    clockMock.Setup(c => c.Now).Returns(new DateTime(2025, 12,
15));

    var calculator = new BonusCalculator(
        repositoryMock.Object, // .Object – собственно объект-
дублёр
        clockMock.Object,
        loggerMock.Object);

    // Act
    decimal bonus = calculator.Calculate(employeeId: 1);

    // Assert – состояние
    Assert.Equal(15_000m, bonus);

    // Assert – взаимодействие (роль Mock)
    loggerMock.Verify(
        l => l.Log(It.Is<string>(s => s.Contains("Иванов"))),
        Times.Once);
}
```

Ключевые элементы Moq:

- `new Mock<T>()` — создание мока интерфейса.
- `Setup(...).Returns(...)` — настройка возвращаемого значения.
- `It.IsAny<T>()`, `It.Is<T>(predicate)` — `matchers` («любое значение», «значение, удовлетворяющее условию»).
- `mock.Object` — собственно объект-реализация интерфейса.
- `Verify(..., Times.Once)` — проверка факта вызова.

6 Разработка через тестирование (TDD)

6.1 Что такое TDD

Test-Driven Development — практика, при которой тест на функциональность пишется раньше кода, реализующего эту функциональность. Каноническое описание дано Кентом Бекем в книге «Экстремальное программирование. Разработка через тестирование».

Важно понимать: TDD — это способ проектирования, а не методология тестирования. Тесты — побочный продукт, главный результат — хорошо спроектированный код. Бек формулирует цель так: «TDD — это про управление страхом во время программирования».

TDD ориентирован прежде всего на разработку бизнес-приложений, его не следует воспринимать как универсальный подход. При разработке, например, вычислительных алгоритмов, которые требуют полной реализации и после этого тщательной отладки и тестирования на большом количестве случаев.

6.2 Цикл *Red — Green — Refactor*

RED → пишем падающий тест на ещё не существующее поведение
 ↓
 GREEN → пишем минимальный код, чтобы тест прошёл
 ↓

REFACTOR → приводим код в порядок (тесты остаются зелёными)

↓

— повторяем цикл для следующего теста

Один цикл — обычно 30 секунд — пара минут. Если фаза занимает заметно больше, то это повод остановиться и пересмотреть подход.

6.3 Три закона TDD (Роберт Мартин)

Сформулированы в статье «The Three Laws of TDD» (<http://butunclebob.com/ArticleS.UncleBob.TheThreeRulesOfTdd>):

1. Не писать продакшен-код, пока нет падающего модульного теста.
2. Не писать кода в тесте больше, чем нужно для того, чтобы он упал.
3. Не писать продакшен-кода больше, чем нужно для прохождения текущего теста.

На практике строго эти правила редко соблюдаются, но они работают как ориентир: если хочется написать «впрок» десять строк без теста — это сигнал вернуться к циклу Red — Green — Refactor.

6.4 Демонстрация: разработка *FamilyFineRule* методом TDD

Заказчик добавляет к *FineCalculator* новую категорию «Семейный абонемент»:

Первые 5 дней — без штрафа.

Далее 2 рубля в день.

Не более 100 рублей всего.

Разрабатываем класс через TDD.

6.4.1 Итерация 1: льготный период

Red. Пишем тест на ещё не существующий класс — он не компилируется (это тоже «падение»):

```
[Fact]
public void Calculate_WithinGracePeriod_ReturnsZero()
{
    var rule = new FamilyFineRule();
    Assert.Equal(0m, rule.Calculate(daysOverdue: 3));
}
```

Green. Создаём класс с минимальной реализацией. Бек называет это «fake it till you make it» — даже захардкоженное значение допустимо, если тест зелёный:

```
public class FamilyFineRule
{
    public decimal Calculate(int daysOverdue) => 0m;
}
```

Refactor. Кода слишком мало — рефакторить нечего.

6.4.2 Итерация 2: оплата после льготного периода

Red. Второй тест заставляет перейти от константы к формуле:

```
[Fact]
public void Calculate_AfterGracePeriod_ChargesTwoRublesPerDay()
{
    var rule = new FamilyFineRule();
    // 8 - 5 льготных = 3 платных дня × 2 рубля = 6
    Assert.Equal(6m, rule.Calculate(daysOverdue: 8));
}
```

Green. Меняем реализацию:

```
public decimal Calculate(int daysOverdue)
{
    if (daysOverdue <= 5) return 0m;
    return (daysOverdue - 5) * 2m;
}
```

Запускаем — оба теста зелёные, рефакторинг безопасен.

Refactor. Можно продолжать вносить изменения в код.

6.4.3 Итоги

- Тесты появились раньше кода — есть гарантия, что каждый из них проверяет именно то поведение, ради которого был написан (он падал!).
- Код вырос постепенно — без оверинжиниринга, без «вдруг пригодится».
- Покрытие — 100% по построению, без специальных усилий.
- Каждый рефакторинг был безопасен: тесты подтверждали неизменность поведения.

6.5 Что даёт TDD кроме тестов

- Дизайн «снаружи внутрь». Чтобы написать тест, нужно сначала придумать API класса. Это улучшает архитектуру.
- Тестируемый дизайн чаще всего эквивалентен соблюдению SOLID. Невозможно делать TDD на коде с нарушениями DIP и SRP — тесты просто не пишутся.
- Естественное соблюдение KISS и YAGNI. Третий закон не позволяет написать ничего лишнего.
- Документация. Финальный набор тестов — это исполняемая спецификация класса.

6.6 Когда TDD неуместен

- **Исследовательская разработка** — когда непонятно, что именно реализовывать. Решение: делать прототип, выбрасывать его, переписывать с TDD.

- **UI-код** — слишком дорого тестировать; бизнес-логику выносят в отдельные классы и тестируют их.
- **Алгоритмы без четкого «правильного ответа»** — математическое моделирование.
- **Одноразовые скрипты** — «стоимость» (время разработки) TDD не окупается.

7 Поведенчески-ориентированная разработка (BDD)

7.1 Что такое BDD

Behavior-Driven Development (BDD) — практика, при которой тесты формулируются на языке, понятном не только программисту, но и бизнес-аналитику или заказчику. Подход предложил Дэн Норт в 2006 году; каноническая статья: «Introducing BDD».

Норт исходил из практической проблемы: разработчики, делающие TDD, тратили много времени на споры о том, что именно тестировать. Слово «test» в имени метода создавало впечатление, что речь о проверке кода — а на деле тесты должны описывать поведение системы с точки зрения пользователя. BDD — это переформулировка TDD на «языке бизнеса».

BDD не отменяет TDD: «под капотом» используется тот же xUnit + Moq. BDD добавляет еще один слой выше — текстовое описание сценариев на полужформальном языке.

7.2 Принцип работы BDD

Сценарии в BDD пишутся на языке Gherkin — изначально созданном для инструмента Cucumber (Ruby) Аслаком Хеллесёем. Gherkin поддерживает русские ключевые слова и работает в .NET, Java, JavaScript, Python и других экосистемах.

Базовая структура — три раздела с ключевыми словами Given / When / Then (по-русски — Дано / Когда / Тогда):

Пример файла «FamilyFineRule.feature»:

language: ru

Функция: Расчёт штрафа для читателей с семейным абонементом

Сценарий: Просрочка в льготный период не штрафуются

Дано читатель с семейным абонементом

Когда книга просрочена на 3 дня

Тогда штраф равен 0 рублей

Сценарий: После льготного периода начисляется 2 рубля в день

Дано читатель с семейным абонементом

Когда книга просрочена на 8 дней

Тогда штраф равен 6 рублей

Сценарий: Штраф ограничен потолком в 100 рублей

Дано читатель с семейным абонементом

Когда книга просрочена на 100 дней

Тогда штраф равен 100 рублей

Структура сценария: Расчёт штрафа для разных периодов

Дано читатель с семейным абонементом

Когда книга просрочена на <дни> дней

Тогда штраф равен <штраф> рублей

Примеры:

дни	штраф
0	0
5	0
6	2
100	100

ВАЖНО! В обозревателе проектов в свойствах файла обязательно необходимо установить тип действия при сборке «feature file».

Тесты из feature-файла через аннотации привязываются к шагам теста:

```
namespace XUnit_BDD
{
    [Binding]
    public class FamilyFineRuleSteps
    {
        private FamilyFineRule? _rule;
        private decimal _actualFine;
```

```

[Given("читатель с семейным абонементом")]
public void GivenFamilyReader()
{
    _rule = new FamilyFineRule();
}

[When(@"книга просрочена на (\d+) дней")]
[When(@"книга просрочена на (\d+) дня")]
public void WhenBookOverdueByDays(int days)
{
    _actualFine = _rule!.Calculate(days);
}

[Then(@"штраф равен (\d+) рублей")]
public void ThenFineEquals(decimal expected)
{
    Assert.Equal(expected, _actualFine);
}
}
}

```

Несколько важных моментов:

1. Атрибут [Binding] обязателен на классе — без него Reqnroll не найдёт привязки.
2. Регулярные выражения в [When] и [Then] извлекают параметры из текста сценария. (\d+) — одна или несколько цифр; Reqnroll сам преобразует строку в int/decimal по типу параметра метода.
3. Дублирование [When] для «дней» и «дня» — простой способ обработать русские падежи. Альтернатива — регулярное выражение «книга просрочена на (\d+) дн(?:я|ей|ь)».
4. Состояние между шагами (_rule, _actualFine) хранится в полях класса. Reqnroll создаёт новый экземпляр класса для каждого сценария (аналогично xUnit), поэтому изоляция сценариев обеспечивается автоматически.

При сборке проекта автоматически будет создан файл с модульным тестами «FamilyFineRule.feature.cs»:

```

// -----
// <auto-generated>
// This code was generated by Reqnroll (https://reqnroll.net/).
// Reqnroll Version:3.0.0.0

```



```
//      Reqnroll Generator Version:3.0.0.0
//
//      Changes to this file may cause incorrect behavior and will be lost if
//      the code is regenerated.
// </auto-generated>
// -----
#region Designer generated code
#pragma warning disable
using Reqnroll;
namespace XUnit_BDD.Features
{

    [global::System.CodeDom.Compiler.GeneratedCodeAttribute("Reqnroll", "3.0.0.0")]
    [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
    public partial class РасчётШтрафаДляЧитателейССемейнымАбонементомFeature : object,
global::Xunit.IClassFixture<РасчётШтрафаДляЧитателейССемейнымАбонементомFeature.FixtureData>,
global::Xunit.IAsyncLifetime
    {

        private global::Reqnroll.ITestRunner testRunner;

        private static string[] featureTags = ((string[])(null));

        private static global::Reqnroll.FeatureInfo featureInfo = new global::Reqnroll.FeatureInfo(new
global::System.Globalization.CultureInfo("ru"), "features", "Расчёт штрафа для читателей с семейным
абонементом", null, global::Reqnroll.ProgrammingLanguage.CSharp, featureTags,
InitializeCucumberMessages());

        private global::Xunit.Abstractions.ITestOutputHelper _testOutputHelper;

#line 1 "FamilyFineRule.feature"
#line hidden

        public
РасчётШтрафаДляЧитателейССемейнымАбонементомFeature(РасчётШтрафаДляЧитателейССемейнымАбонементомFeature
.FixtureData fixtureData, global::Xunit.Abstractions.ITestOutputHelper testOutputHelper)
        {
            this._testOutputHelper = testOutputHelper;
        }

        public static async global::System.Threading.Tasks.Task FeatureSetupAsync()
        {
        }

        public static async global::System.Threading.Tasks.Task FeatureTearDownAsync()
        {
            await global::Reqnroll.TestRunnerManager.ReleaseFeatureAsync(featureInfo);
        }

        public async global::System.Threading.Tasks.Task TestInitializeAsync()
        {
            testRunner = global::Reqnroll.TestRunnerManager.GetTestRunnerForAssembly(featureHint:
featureInfo);
            try
            {
                if (((testRunner.FeatureContext != null)
                    && (testRunner.FeatureContext.FeatureInfo.Equals(featureInfo) == false)))
                {
                    await testRunner.OnFeatureEndAsync();
                }
            }
            finally
            {
                if (((testRunner.FeatureContext != null)
                    && testRunner.FeatureContext.BeforeFeatureHookFailed))
                {
                    throw new global::Reqnroll.ReqnrollException("Scenario skipped because of previous
before feature hook error");
                }
                if ((testRunner.FeatureContext == null))
                {
                    await testRunner.OnFeatureStartAsync(featureInfo);
                }
            }
        }
    }
}
```

```

    }

    public async global::System.Threading.Tasks.Task TestTearDownAsync()
    {
        if ((testRunner == null))
        {
            return;
        }
        try
        {
            await testRunner.OnScenarioEndAsync();
        }
        finally
        {
            global::Reqnroll.TestRunnerManager.ReleaseTestRunner(testRunner);
            testRunner = null;
        }
    }

    public void ScenarioInitialize(global::Reqnroll.ScenarioInfo scenarioInfo,
global::Reqnroll.RuleInfo ruleInfo)
    {
        testRunner.OnScenarioInitialize(scenarioInfo, ruleInfo);

        testRunner.ScenarioContext.ScenarioContainer.RegisterInstanceAs<global::Xunit.Abstractions.ITestOutputH
elper>(_testOutputHelper);
    }

    public async global::System.Threading.Tasks.Task ScenarioStartAsync()
    {
        await testRunner.OnScenarioStartAsync();
    }

    public async global::System.Threading.Tasks.Task ScenarioCleanupAsync()
    {
        await testRunner.CollectScenarioErrorsAsync();
    }

    private static global::Reqnroll.Formatter.RuntimeSupport.FeatureLevelCucumberMessages
InitializeCucumberMessages()
    {
        return new
global::Reqnroll.Formatter.RuntimeSupport.FeatureLevelCucumberMessages("features/FamilyFineRule.featur
e.ndjson", 9);
    }

    async global::System.Threading.Tasks.Task global::Xunit.IAsyncLifetime.InitializeAsync()
    {
        try
        {
            await this.TestInitializeAsync();
        }
        catch (System.Exception e1)
        {
            try
            {
                ((global::Xunit.IAsyncLifetime)(this)).DisposeAsync();
            }
            catch (System.Exception e2)
            {
                throw new System.AggregateException("Test initialization failed", e1, e2);
            }
            throw;
        }
    }

    async global::System.Threading.Tasks.Task global::Xunit.IAsyncLifetime.DisposeAsync()
    {
        await this.TestTearDownAsync();
    }

    [global::Xunit.SkippableFactAttribute(DisplayName="Просрочка в льготный период не штрафуетс")]
    [global::Xunit.TraitAttribute("FeatureTitle", "Расчёт штрафа для читателей с семейным
абонементом")]
    [global::Xunit.TraitAttribute("Description", "Просрочка в льготный период не штрафуетс")]

```

```

public async global::System.Threading.Tasks.Task ПросрочкаВЛьготныйПериодНеШтрафуется()
{
    string[] tagsOfScenario = ((string[])(null));
    global::System.Collections.Specialized.OrderedDictionary argumentsOfScenario = new
global::System.Collections.Specialized.OrderedDictionary();
    string pickleIndex = "0";
    global::Reqnroll.ScenarioInfo scenarioInfo = new global::Reqnroll.ScenarioInfo("Просрочка в
льготный период не штрафуются", null, tagsOfScenario, argumentsOfScenario, featureTags, pickleIndex);
    string[] tagsOfRule = ((string[])(null));
    global::Reqnroll.RuleInfo ruleInfo = null;

#line 4
    this.ScenarioInitialize(scenarioInfo, ruleInfo);
#line hidden
    if ((global::Reqnroll.TagHelper.ContainsIgnoreTag(scenarioInfo.CombinedTags) ||
global::Reqnroll.TagHelper.ContainsIgnoreTag(featureTags)))
    {
        await testRunner.SkipScenarioAsync();
    }
    else
    {
        await this.ScenarioStartAsync();

#line 5
        await testRunner.GivenAsync("читатель с семейным абонементом", ((string)(null)),
((global::Reqnroll.Table)(null)), "Дано ");
#line hidden
#line 6
        await testRunner.WhenAsync("книга просрочена на 3 дня", ((string)(null)),
((global::Reqnroll.Table)(null)), "Когда ");
#line hidden
#line 7
        await testRunner.ThenAsync("штраф равен 0 рублей", ((string)(null)),
((global::Reqnroll.Table)(null)), "Тогда ");
#line hidden
    }
    await this.ScenarioCleanupAsync();
}

[global::Xunit.SkippableFactAttribute(DisplayName="После льготного периода начисляется 2 рубля
в день")]
[global::Xunit.TraitAttribute("FeatureTitle", "Расчёт штрафа для читателей с семейным
абонементом")]
[global::Xunit.TraitAttribute("Description", "После льготного периода начисляется 2 рубля в
день")]
public async global::System.Threading.Tasks.Task ПослеЛьготногоПериодаНачисляется2РубляВДень()
{
    string[] tagsOfScenario = ((string[])(null));
    global::System.Collections.Specialized.OrderedDictionary argumentsOfScenario = new
global::System.Collections.Specialized.OrderedDictionary();
    string pickleIndex = "1";
    global::Reqnroll.ScenarioInfo scenarioInfo = new global::Reqnroll.ScenarioInfo("После
льготного периода начисляется 2 рубля в день", null, tagsOfScenario, argumentsOfScenario, featureTags,
pickleIndex);
    string[] tagsOfRule = ((string[])(null));
    global::Reqnroll.RuleInfo ruleInfo = null;

#line 9
    this.ScenarioInitialize(scenarioInfo, ruleInfo);
#line hidden
    if ((global::Reqnroll.TagHelper.ContainsIgnoreTag(scenarioInfo.CombinedTags) ||
global::Reqnroll.TagHelper.ContainsIgnoreTag(featureTags)))
    {
        await testRunner.SkipScenarioAsync();
    }
    else
    {
        await this.ScenarioStartAsync();

#line 10
        await testRunner.GivenAsync("читатель с семейным абонементом", ((string)(null)),
((global::Reqnroll.Table)(null)), "Дано ");
#line hidden
#line 11
        await testRunner.WhenAsync("книга просрочена на 8 дней", ((string)(null)),
((global::Reqnroll.Table)(null)), "Когда ");
#line hidden
#line 12

```

```

        await testRunner.ThenAsync("штраф равен 6 рублей", ((string)(null)),
((global::Reqnroll.Table)(null)), "Тогда ");
#line hidden
    }
    await this.ScenarioCleanupAsync();
}

[global::Xunit.SkippableFactAttribute(DisplayName="Штраф ограничен потолком в 100 рублей")]
[global::Xunit.TraitAttribute("FeatureTitle", "Расчёт штрафа для читателей с семейным
абонементом")]
[global::Xunit.TraitAttribute("Description", "Штраф ограничен потолком в 100 рублей")]
public async global::System.Threading.Tasks.Task ШтрафОграниченПотолкомВ100Рублей()
{
    string[] tagsOfScenario = ((string[])(null));
    global::System.Collections.Specialized.OrderedDictionary argumentsOfScenario = new
global::System.Collections.Specialized.OrderedDictionary();
    string pickleIndex = "2";
    global::Reqnroll.ScenarioInfo scenarioInfo = new global::Reqnroll.ScenarioInfo("Штраф
ограничен потолком в 100 рублей", null, tagsOfScenario, argumentsOfScenario, featureTags, pickleIndex);
    string[] tagsOfRule = ((string[])(null));
    global::Reqnroll.RuleInfo ruleInfo = null;
#line 14
    this.ScenarioInitialize(scenarioInfo, ruleInfo);
#line hidden
    if ((global::Reqnroll.TagHelper.ContainsIgnoreTag(scenarioInfo.CombinedTags) ||
global::Reqnroll.TagHelper.ContainsIgnoreTag(featureTags)))
    {
        await testRunner.SkipScenarioAsync();
    }
    else
    {
        await this.ScenarioStartAsync();
#line 15
        await testRunner.GivenAsync("читатель с семейным абонементом", ((string)(null)),
((global::Reqnroll.Table)(null)), "Дано ");
#line hidden
#line 16
        await testRunner.WhenAsync("книга просрочена на 100 дней", ((string)(null)),
((global::Reqnroll.Table)(null)), "Когда ");
#line hidden
#line 17
        await testRunner.ThenAsync("штраф равен 100 рублей", ((string)(null)),
((global::Reqnroll.Table)(null)), "Тогда ");
#line hidden
    }
    await this.ScenarioCleanupAsync();
}

[global::Xunit.SkippableTheoryAttribute(DisplayName="Расчёт штрафа для разных периодов")]
[global::Xunit.TraitAttribute("FeatureTitle", "Расчёт штрафа для читателей с семейным
абонементом")]
[global::Xunit.TraitAttribute("Description", "Расчёт штрафа для разных периодов")]
[global::Xunit.InlineDataAttribute("0", "0", "3", new string[0])]
[global::Xunit.InlineDataAttribute("5", "0", "4", new string[0])]
[global::Xunit.InlineDataAttribute("6", "2", "5", new string[0])]
[global::Xunit.InlineDataAttribute("100", "100", "6", new string[0])]
public async global::System.Threading.Tasks.Task РасчётШтрафаДляРазныхПериодов(string дни,
string штраф, string @__pickleIndex, string[] exampleTags)
{
    string[] tagsOfScenario = exampleTags;
    global::System.Collections.Specialized.OrderedDictionary argumentsOfScenario = new
global::System.Collections.Specialized.OrderedDictionary();
    argumentsOfScenario.Add("дни", дни);
    argumentsOfScenario.Add("штраф", штраф);
    string pickleIndex = @__pickleIndex;
    global::Reqnroll.ScenarioInfo scenarioInfo = new global::Reqnroll.ScenarioInfo("Расчёт
штрафа для разных периодов", null, tagsOfScenario, argumentsOfScenario, featureTags, pickleIndex);
    string[] tagsOfRule = ((string[])(null));
    global::Reqnroll.RuleInfo ruleInfo = null;
#line 19
    this.ScenarioInitialize(scenarioInfo, ruleInfo);
#line hidden
    if ((global::Reqnroll.TagHelper.ContainsIgnoreTag(scenarioInfo.CombinedTags) ||
global::Reqnroll.TagHelper.ContainsIgnoreTag(featureTags)))
    {

```

```

        await testRunner.SkipScenarioAsync();
    }
    else
    {
        await this.ScenarioStartAsync();
#line 20
        await testRunner.GivenAsync("читатель с семейным абонементом", ((string)(null)),
((global::Reqnroll.Table)(null)), "Дано ");
#line hidden
#line 21
        await testRunner.WhenAsync(string.Format("книга просрочена на {0} дней", дни), ((string)(null)),
((global::Reqnroll.Table)(null)), "Когда ");
#line hidden
#line 22
        await testRunner.ThenAsync(string.Format("штраф равен {0} рублей", штраф), ((string)(null)),
((global::Reqnroll.Table)(null)), "Тогда ");
#line hidden
    }
    await this.ScenarioCleanupAsync();
}

[global::System.CodeDom.Compiler.GeneratedCodeAttribute("Reqnroll", "3.0.0.0")]
[global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
public class FixtureData : object, global::Xunit.IAsyncLifetime
{
    async global::System.Threading.Tasks.Task global::Xunit.IAsyncLifetime.InitializeAsync()
    {
        await РасчётШтрафаДляЧитателейССемейнымАбонементомFeature.FeatureSetupAsync();
    }

    async global::System.Threading.Tasks.Task global::Xunit.IAsyncLifetime.DisposeAsync()
    {
        await РасчётШтрафаДляЧитателейССемейнымАбонементомFeature.FeatureTearDownAsync();
    }
}
}
}
#pragma warning restore
#endregion

```

Тесты будут отображены в обозревателе тестов.

Тестирование	Длительность	Признаки	Сообщение об ошибке
> MStest_Console (5)	65 мс		
> NUnit_Test_Console (5)	17 мс		
> XUnit_BDD (7)	125 мс		
> XUnit_BDD.Features (7)	125 мс		
> РасчётШтрафаДляЧитателейССемейнымАбонементомFeature (7)	125 мс		
> РасчётШтрафаДляРазныхПериодов (4)	118 мс		
> Расчёт штрафа для разных периодов(дни: "0", штраф: "0", __pickleIndex: "3", exampleTags: [])	3 мс	FeatureTitl...	
> Расчёт штрафа для разных периодов(дни: "100", штраф: "100", __pickleIndex: "6", exampleTags: [])	< 1 мс	FeatureTitl...	
> Расчёт штрафа для разных периодов(дни: "5", штраф: "0", __pickleIndex: "4", exampleTags: [])	115 мс	FeatureTitl...	
> Расчёт штрафа для разных периодов(дни: "6", штраф: "2", __pickleIndex: "5", exampleTags: [])	< 1 мс	FeatureTitl...	
> После льготного периода начисляется 2 рубля в день	2 мс	FeatureTitl...	
> Просрочка в льготный период не штрафует	3 мс	FeatureTitl...	
> Штраф ограничен потолком в 100 рублей	2 мс	FeatureTitl...	
> XUnit_Test_Console (5)	16 мс		

Рис. 12. BDD-тесты в обозревателе тестов.

Для логики «до/после сценария», аналогичной [TestInitialize]/[TestCleanup], Reqnroll предоставляет hooks:

```

[Binding]
public class TestHooks
{
    [BeforeScenario]
    public void BeforeEachScenario() { /* подготовка */ }

    [AfterScenario]
    public void AfterEachScenario() { /* очистка */ }

    [BeforeFeature]
    public static void BeforeFeature() { /* один раз перед всеми
сценариями функции */ }

    [AfterTestRun]
    public static void AfterAllTests() { /* один раз после всех тестов
*/ }
}

```

8 Тестирование пользовательского интерфейса

UI-тесты находятся на вершине пирамиды тестирования (раздел 2.3): они медленные, хрупкие и дорогие в поддержке. Применяются для проверки критичных сквозных пользовательских сценариев, которых должны быть единицы — основная масса логики покрывается юнит- и интеграционными тестами.

В .NET сложились следующие инструменты для трёх типов приложений:

Тип приложения	Инструмент	Лицензия
• WinForms	фреймворк FlaUI	
• WPF	фреймворк FlaUI	
• ASP.NET Core MVC	фреймворк Playwright for .NET	

8.1 WinForms с FlaUI

FlaUI (<https://github.com/FlaUI/FlaUI>) — обёртка над Microsoft UI Automation, штатной технологией Windows для автоматизации UI.

Пример теста:

```

using FlaUI.Core;
using FlaUI.UIA3;
using Xunit;

[Fact]
public void AddingBook_DisplaysItInList()
{
    using var app = Application.Launch("LibraryApp.WinForms.exe");
    using var automation = new UIA3Automation();
    var window = app.GetMainWindow(automation);

    // Заполняем форму
    window.FindFirstDescendant(cf =>
cf.ByAutomationId("titleTextBox"))
        .AsTextBox().Enter("Евгений Онегин");
    window.FindFirstDescendant(cf =>
cf.ByAutomationId("authorTextBox"))
        .AsTextBox().Enter("Пушкин");
    window.FindFirstDescendant(cf => cf.ByAutomationId("addButton"))
        .AsButton().Invoke();

    // Проверяем
    var list = window.FindFirstDescendant(cf =>
cf.ByAutomationId("booksListBox"))
        .AsListBox();
    Assert.Contains(list.Items, item => item.Text.Contains("Евгений
Онегин"));
}

```

8.2 WPF с FlaUI

Тот же FlaUI без изменений работает с WPF — WPF поддерживает UI Automation «из коробки». Для элементов задаётся свойство AutomationProperties.AutomationId:

```

<!-- В XAML тестируемого WPF-окна -->
<TextBox AutomationProperties.AutomationId="titleTextBox" />
<Button AutomationProperties.AutomationId="addButton"
Content="Добавить" />

```

Тестовый код практически идентичен примеру для WinForms:

```
[Fact]
public void AddingBook_AppearsInDataGrid()
{
    using var app = Application.Launch("LibraryApp.Wpf.exe");
    using var automation = new UIA3Automation();
    var window = app.GetMainWindow(automation);

    window.FindFirstDescendant(cf =>
cf.ByAutomationId("titleTextBox"))
        .AsTextBox().Enter("Война и мир");
    window.FindFirstDescendant(cf => cf.ByAutomationId("addButton"))
        .AsButton().Invoke();

    var grid = window.FindFirstDescendant(cf =>
cf.ByAutomationId("booksGrid"))
        .AsDataGridview();
    Assert.Contains(grid.Rows, r => r.Cells[0].Value?.ToString() ==
"Война и мир");
}
```

Важно отметить, что в WPF/WinForms-приложениях правильно построенная MVVM/MVP-архитектура позволяет покрывать всю бизнес-логику обычными юнит-тестами, тестируя ViewModel без участия UI. UI-тесты тогда нужны только для проверки, что разметка корректно связана с ViewModel.

8.3 ASP.NET Core MVC с Playwright

Playwright for .NET (<https://playwright.dev/dotnet/>) — современный кросс-браузерный инструмент от Microsoft, поддерживает Chromium, Firefox и WebKit. Пришёл на смену Selenium как более быстрый и стабильный.

Пример теста для веб-приложения:

```
using Microsoft.Playwright;
using Xunit;

public class LibraryWebTests : IAsyncLifetime
{
    private IPlaywright _playwright = null!;
```



```

private IBrowser _browser = null!;

public async Task InitializeAsync()
{
    _playwright = await Playwright.CreateAsync();
    _browser = await _playwright.Chromium.LaunchAsync(
        new() { Headless = true });
}

public async Task DisposeAsync()
{
    await _browser.DisposeAsync();
    _playwright.Dispose();
}

[Fact]
public async Task AddBook_ShowsInCatalog()
{
    var page = await _browser.NewPageAsync();
    await page.GotoAsync("http://localhost:5000/books/add");

    await page.FillAsync("#Title", "Мастер и Маргарита");
    await page.FillAsync("#Author", "Булгаков");
    await page.ClickAsync("button[type=submit]");

    await page.WaitForURLAsync("*/books");
    await Assertions.Expect(page.Locator("table"))
        .ToContainTextAsync("Мастер и Маргарита");
}
}

```

9 Основные выводы

- Тесты — обязательная часть профессиональной разработки, а не «дополнительная нагрузка». Они окупаются уже на стадии написания кода за счёт более быстрой обратной связи, и тем более — при поддержке и рефакторинге.
- Тестируемость — следствие хорошего дизайна. Класс, спроектированный по SOLID и с внедрением зависимостей, тестируется тривиально. Класс, нарушающий эти принципы, тестируется тяжело или никак. Сложности с написанием тестов — это сигнал о проблемах архитектуры, а не о проблемах тестирования.

- Пирамида тестирования распределяет роли по уровням: основная масса тестов — модульные (быстрые, изолированные), меньше — интеграционные, ещё меньше — сквозные. Перевернутая пирамида (много E2E, мало юнитов) — типичная антипрактика, приводящая к долгим и нестабильным сборкам.
- Среди трёх фреймворков (MSTest, NUnit, xUnit) для новых проектов на .NET рекомендуется xUnit как современный стандарт. MSTest и NUnit функционально эквивалентны и применяются в командах, где они уже укоренились.
- Тестовые дублёры (Dummy, Stub, Fake, Mock, Spy) — инструмент изоляции тестируемого класса от внешнего мира. Библиотеки Moq или NSubstitute автоматизируют их создание. Главное правило — проверять моками только наблюдаемые снаружи побочные эффекты, а не детали реализации.
- TDD — это не «способ писать тесты», а способ проектировать код. Цикл Red — Green — Refactor даёт постепенный рост системы под постоянной защитой тестов. Главный результат TDD — хороший дизайн; тесты — приятный побочный эффект.
- BDD — расширение TDD, переводящее формулировку требований на язык бизнеса. Их следует применять тогда, когда есть кому читать .feature-файлы кроме программиста. Без процесса совместной работы с аналитиками BDD вырождается в дорогой TDD.
- Покрытие кода — индикатор, а не цель. 100 % покрытия плохими тестами хуже, чем 70 % хорошими.
- UI-тесты дорогие и хрупкие. Их должно быть мало; основная логика приложения покрывается ниже по пирамиде. Архитектуры MVVM и MVC специально нужны в том числе для того, чтобы

большую часть логики UI-приложений можно было тестировать обычными юнит-тестами.